Chapter 7: Linked-Lists

Objectives:

- Various types of Linked-Lists
- The positional list ADT
- Sorting a positional list
- Case study: Maintaining Access
- Frequencies
- Link-Based Vs Array-based Sequences

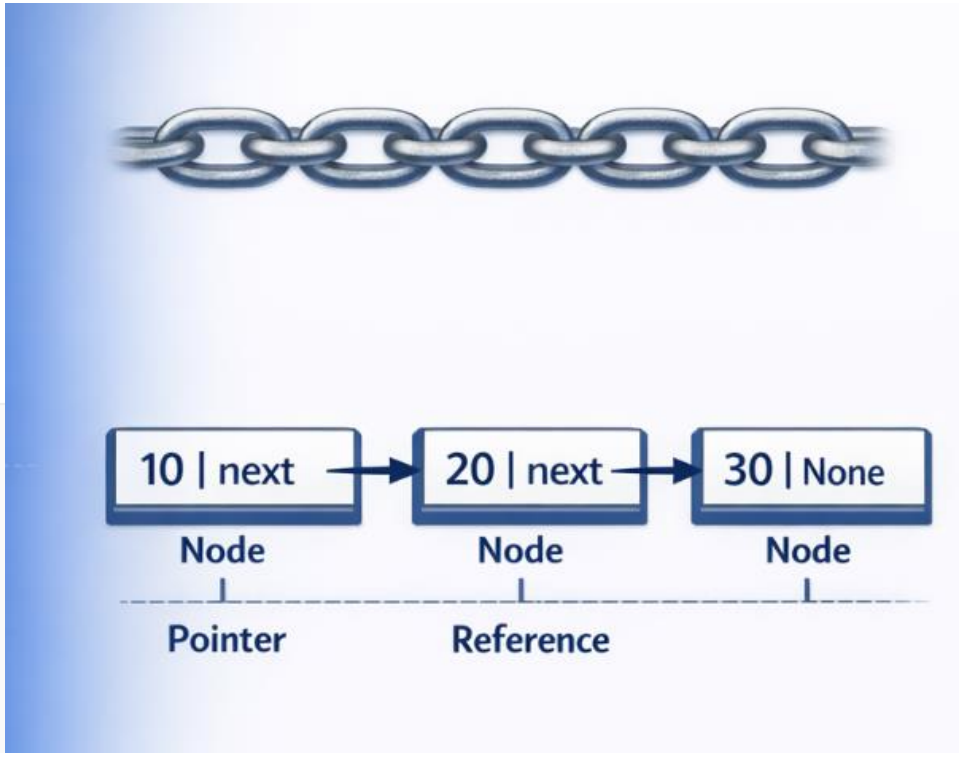
1: Introduction to Linked Lists

📌 Definition

A **Linked List** is a linear data structure where elements (nodes) are connected using pointers.

Each node contains:

- Data
- Reference (link) to next node



💻 Simple Node in Python

‹/› Python

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

2: Why Linked Lists?

📌 Problems with Arrays

- Fixed size
- Costly insertion/deletion (shifting elements)

📌 Advantages of Linked List

- Dynamic size
- Efficient insert/delete at head

Array insertion:

‹/› **Code**

```
[10][20][30]
Insert 15 → shift required
```

Linked List insertion:

‹/› **Code**

```
10 → 20 → 30
Insert 15 between 10 and 20
```

Various Types of Linked Lists
3: Singly Linked List

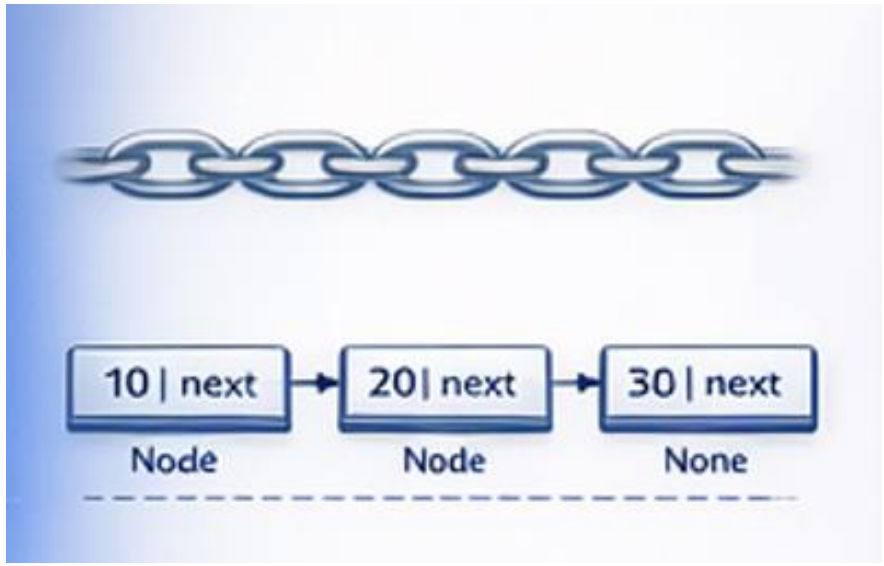
Each node points to next node only.

💻 Example

‹/› Python

```python
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def add_first(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```




```
Head → [10] → [20] → [30] → None
```

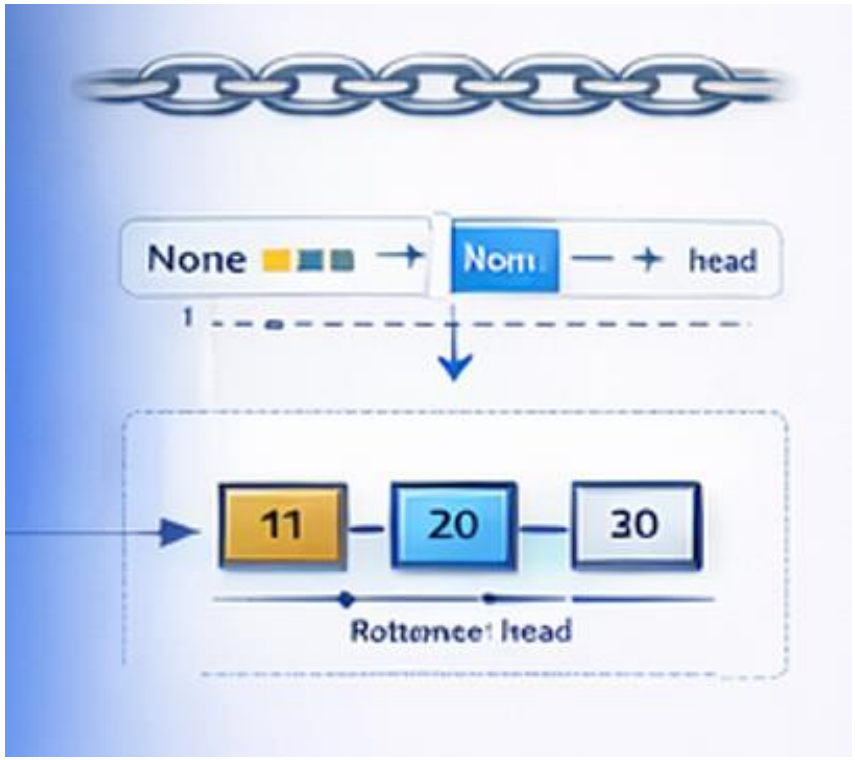
4: Doubly Linked List

Each node has:

- next pointer
- prev pointer

💻 Example

```python
class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```




```
None ← [10] ⇄ [20] ⇄ [30] → None
```

Bidirectional movement.

5: Circular Linked List

Last node points back to first node.

💻 Example

‹/› Python

```python
tail.next = head
```

🖼️ Image Explanation

‹/› Code

```
10 → 20 → 30
↑          ↓
← ← ← ← ←
```

Used in:

- Round-robin scheduling

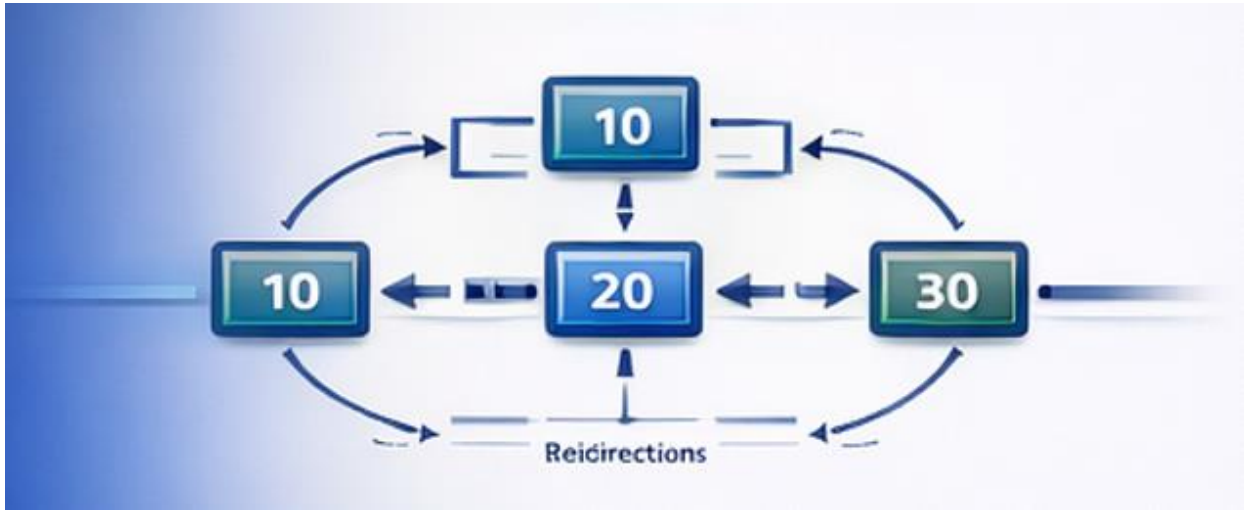
6: Circular Doubly Linked List

Both directions circular.

🖼️ Image Explanation

‹/› Code

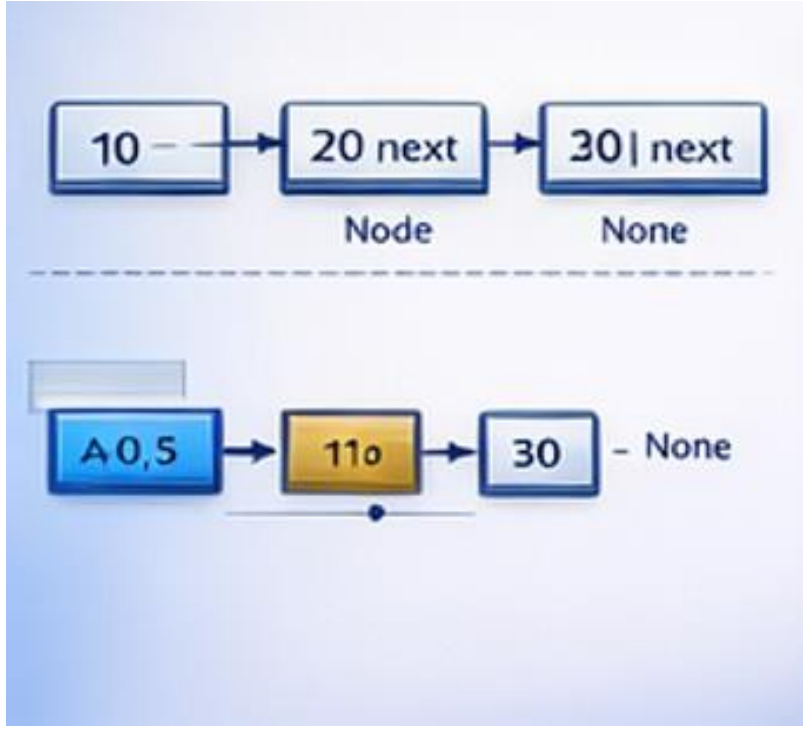
```
  ← ← ← ← ←
10 ⇄ 20 ⇄ 30
  → → → → →
```



Positional List ADT

7: What is Positional List?

- Abstract Data Type that:
  - Stores elements
  - Provides position abstraction
- Position ≠ index
  Position = location object



8: Position Class

💻 Example

‹/› Python

```python
class Position:
    def __init__(self, container, node):
        self.container = container
        self.node = node

    def element(self):
        return self.node.data
```

🖼️ Image Explanation

‹/› Code

```
Position → Node → Data
```

Position acts like a wrapper.

9: Insert Between Nodes

💻 Code

‹/› Python

```python
def insert_between(self, data, predecessor, successor):
    new = DNode(data)
    new.prev = predecessor
    new.next = successor
    predecessor.next = new
    successor.prev = new
```

Before:

‹/› Code

```
10 ⇄ 30
```

After inserting 20:

‹/› Code

```
10 ⇄ 20 ⇄ 30
```

🔹 Sorting a Positional List
10: Insertion Sort on Linked List

💻 Code

‹/› Python

```python
def insertion_sort(L):
    if len(L) > 1:
        marker = L.first()
        while marker != L.last():
            pivot = L.after(marker)
            value = pivot.element()
            if value >= marker.element():
                marker = pivot
            else:
                walk = marker
                while walk != L.first() and L.before(walk).element() > value:
                    walk = L.before(walk)
                L.delete(pivot)
                L.add_before(walk, value)
```

11: Sorting Visualization

Before:

Code

```
30 → 10 → 20
```

Step 1:

Insert 10 before 30

Step 2:

Insert 20 between 10 and 30

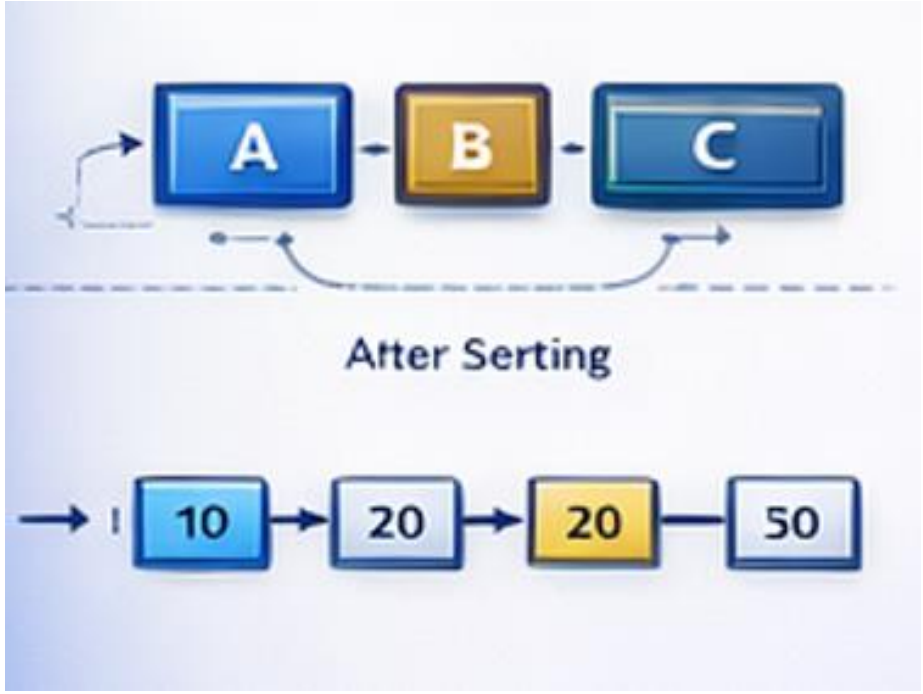
After:

Code

```
10 → 20 → 30
```

Case Study: Maintaining Access Frequencies 12: Access Frequency Problem

- We want:
  - Frequently accessed items move forward.
- Used in:
  - Caching
  - Database systems



13: Self-Organizing List (Move-To-Front)

💻 Code

```python
def access(self, p):
    self._data.delete(p)
    self._data.add_first(p.element())
```

Before access:

</> Code

A → B → C → D

Access C:

</> Code

C → A → B → D

14: Count Method

Store frequency in node.

💻 Example

‹/› Python

```python
class FreqNode:
    def __init__(self, data):
        self.data = data
        self.count = 0
```

Sort by highest count.

15: Frequency Example

Access pattern:

A, B, A, C, A

Frequency:

A=3, B=1, C=1

List:

Code

A → B → C

Link-Based vs Array-Based Sequences

16: Array-Based Sequences

Example: Python list

```python
arr = [10, 20, 30]
```

```
Index: 0   1   2
       10  20  30
```

Contiguous memory.

17: Link-Based Sequences

‹/› Python

```python
head = Node(10)
head.next = Node(20)
```

🖼️ Image Explanation

‹/› Code

```
[10] → [20] → [30]
```

Scattered memory.

18: Comparison Table

Feature	Array	Linked List
Access	O(1)	O(n)
Insert at head	O(n)	O(1)
Memory	Contiguous	Non-contiguous
Resize	Costly	Dynamic

19: Time Complexity Summary

Operation	Singly LL	Doubly LL
Insert head	O(1)	O(1)
Insert tail	O(n)	O(1)*
Search	O(n)	O(n)
Delete	O(n)	O(1)*

* If node reference is known.

🧪 Lab 1: Implement Singly Linked List

🎯 Objective:

Create a singly linked list with:

- Insert at beginning
- Insert at end
- Display list

📝 Practice Input:

Insert: 10, 20, 30

Expected Output:

```
10 → 20 → 30 → None
```

🧪 Lab 2: Delete Node from Linked List

🎯 Objective:

Delete:

- First node
- Last node
- Node by value

📝 Test Case:

Before:

‹/› **Code**

```
10 → 20 → 30 → None
```

Delete 20

After:

‹/› **Code**

```
10 → 30 → None
```

🧪 Lab 3: Implement Doubly Linked List

🎯 **Objective:**

Create:

- Insert at beginning
- Traverse forward and backward

💻 **Task:**

‹/› Python

```python
class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

Students must:

- Maintain both next and prev pointers
- Print forward and backward

🧪 Lab 4: Reverse a Singly Linked List

- 🎯 **Objective:**
- Reverse the linked list

📝 **Example:**

Before:

```
Code

10 → 20 → 30 → None
```

After:

```
Code

30 → 20 → 10 → None
```

🧪 Lab 5: Count Frequency of Elements

🎯 **Objective:**

Count how many times each value appears.

💻 **Task:**

‹/› Python

```python
def count_frequency(self):
    freq = {}
    temp = self.head
    while temp:
        freq[temp.data] = freq.get(temp.data, 0) + 1
        temp = temp.next
    return freq
```

📝 **Input:**

‹/› Code

```
10 → 20 → 10 → 30 → 20
```

Output:

‹/› Code

```
{10:2, 20:2, 30:1}
```

📌 Assignment 1: Student Record Management System (Using Linked List)

🎯 Description:

Create a system that stores student records using a linked list.

Each Node Contains:

- Student ID
- Name
- GPA

Required Operations:

1. Add student
2. Delete student
3. Search student
4. Display all students
5. Sort students by GPA

📊 Evaluation Criteria:

Criteria	Marks
Correct Linked List Implementation	30
Sorting Algorithm	20
Menu-driven Interface	20
Code Structure	15
Output Format	15